

Introduction

This is the exercise sheet for Lecture 2. During the lecture you saw the core building blocks of the Entity-Relationship (ER) model: entities, attributes (simple, composite, multi-valued, derived), relationships, cardinality, participation, and weak entities with their identifying relationships. You also saw a live **Order–OrderLine** example for weak entities, and an extended library-system demo drawn live in Mermaid/draw.io. In this session you will:

- warm up by picking entities, attributes, and a relationship out of a short plain-English scenario,
- classify attributes as simple, composite, multi-valued, or derived,
- practice weak entities and identifying relationships on a new example (not the lecture’s **Order–OrderLine** one) that also adds an extra entity to test whether you can tell a weak entity apart from an ordinary one on the “many” side of a 1:N relationship,
- spot and fix the problems in a flawed, informally-described ER model,
- design a full ER model for a library system,
- design a larger ER model for a university, combining everything from earlier in the sheet – weak entities, a relationship promoted to an entity, and a rare 1:1 relationship, and
- practice ER modelling on a domain of your choice (e.g. your semester project).

You can find ready-to-view/edit diagram sources (PlantUML `.puml` files, with rendered `.svg` images alongside them) for the library system in this sheet’s own **figures/** folder, and for the lecture’s **Order–OrderLine** example in the **extras/** folder (see **How-to-Draw-ER-Diagrams.md** in **extras/**); Exercise 3 below uses a different domain of its own, with no pre-drawn diagram to check against.

1 Exercise 1: Warm-up – Entities, Attributes, and a Relationship

Read the following short scenario:

A car rental company rents out cars to customers. Each car has a licence plate, a model name, and a daily rental price. Each customer has a name and a driver’s licence number. When a customer rents a car, the company records the pickup date and the return date for that rental.

Answer the following, in words – no diagram needed yet:

1. Which nouns in the scenario are *entities* (things with their own identity, about which we store data)?

2. For each entity you identified, which nouns/phrases are its *attributes* (properties of that entity)?
3. Spot one *relationship* in the scenario: which two entities does it connect, and what would you name it?

Solution.

1. Two entities: *Car* and *Customer*. (“Rental” is arguably a third candidate, but at this stage it is most naturally modelled as a relationship between *Car* and *Customer* – see part 3.)
2. *Car*’s attributes: licence plate, model name, daily rental price. *Customer*’s attributes: name, driver’s licence number.
3. The relationship connects *Car* and *Customer*; a natural name is *Rents*. The pickup date and return date belong to this relationship (a particular car being rented by a particular customer on a particular occasion), not to *Car* or *Customer* alone – an example of an attribute that lives on the relationship itself, as seen with *Borrows* in the library example.

2 Exercise 2: Classify the Attributes

Consider a **Lecturer** entity in a university database, with the following pieces of information recorded about each lecturer:

- **staff_id**
- **full_name**, made up of a **first_name** and a **last_name**
- **email**
- **office_phone_numbers** (a lecturer may have more than one office phone number, e.g. at different campuses)
- **years_of_service**, calculated from a stored **start_date** and the current date
- **start_date**

For each attribute above, classify it as **simple**, **composite**, **multi-valued**, or **derived**. (An attribute can only belong to one of the first three categories, but derived is a separate, orthogonal label some textbooks apply on top of the others – here just pick the single best-fitting label from the four.)

Solution.

- **staff_id** – **simple**: it is a single, atomic value.
- **full_name** – **composite**: it is made up of sub-parts, **first_name** and **last_name**.
- **email** – **simple**: one atomic value (we are told nothing about a lecturer having several email addresses here).
- **office_phone_numbers** – **multi-valued**: a lecturer can have more than one of these at the same time.
- **years_of_service** – **derived**: it is computed from another stored attribute (**start_date**) and the current date, rather than stored directly – exactly like **is_overdue** for a loan in the library example.
- **start_date** – **simple**: a single atomic date value.

3 Exercise 3: Weak Entities – Course, Topic, and Professor

This is a different domain than the lecture's **Order–OrderLine** example, with one extra entity added to make the reasoning less automatic. A **Course** has a `course_id` and a `title`. Each course is organized into one or more weekly **Topics**; each **Topic** records a `topic_number`, a `topic_title`, and a `duration_minutes`, but a topic only makes sense together with the course it belongs to, and a topic is only numbered uniquely *within* its own course (topic 1, topic 2, ... – these numbers repeat across different courses). Every course also has exactly one **Professor** responsible for teaching it, identified by `professor_id` and `name`; the same professor can be responsible for several different courses.

Answer the following:

1. Which of the three entities – **Course**, **Topic**, or **Professor** – is the *weak entity*? Justify your answer in one sentence.
2. What is **Topic**'s *partial key* (discriminator)? Why is it not, on its own, enough to uniquely identify a topic in the whole database?
3. Describe the *identifying relationship* between **Course** and **Topic**: what would you call it, what is its cardinality, and what is the participation constraint on each side?
4. **Course** is also connected to **Professor** by a **1:N** relationship (one professor, many courses) – the same cardinality shape as the identifying relationship in Question 3. Is this **Professor–Course** relationship identifying, and is **Course** a weak entity with respect to **Professor**? Justify your answer, and state the general rule for telling the two situations apart.

Solution.

1. *Topic is the weak entity: it has no attribute of its own that uniquely identifies it across the whole database, and it cannot exist without being linked to an owning Course. Course and Professor both have their own id (course_id, professor_id) and are strong entities.*
2. *Topic's partial key is topic_number. On its own it is not enough, because topic_number only distinguishes topics within the same course – topic 1 of one course and topic 1 of another course are two different rows. Topic's real (composite) key is therefore (course_id, topic_number), combining the owner's key with the partial key.*
3. *The identifying relationship is, e.g., Covers, between Course and Topic, with cardinality 1:N (one course covers many topics, one topic belongs to exactly one course). Participation is total on both sides: a topic cannot exist without its course (drawn with a double line to Topic in ER notation), and – since the scenario states each course is organized into one or more topics – a course likewise requires at least one topic, so Course's participation is total too.*

4. No on both counts. *Teaches* (*Professor–Course*) is an ordinary, **non-identifying** 1:N relationship, and *Course* is **not** weak with respect to *Professor*: *Course* already has its own key, *course_id*, that uniquely identifies it across the whole database with no help from *Professor*. **General rule:** being on the “many” side of a 1:N relationship is necessary but nowhere near sufficient for weakness – most 1:N relationships (like *Teaches* here) connect two perfectly ordinary strong entities. An entity is weak only if both hold: (a) it has no attribute or attribute combination of its own that is globally unique, and (b) it cannot be identified, even in principle, without referring to the owning entity’s key – exactly the situation *Topic* is in with respect to *Course*, and *Course* is not in with respect to *Professor*.

4 Exercise 4: Spot the Problem – A Flawed Car-Rental ER Model

A classmate modelled a slightly bigger version of the car-rental scenario (building on Exercise 1) and described their ER model to you informally, as follows:

“I have an entity **Car** with attributes `licence_plate`, `model_name`, and `daily_price`. I have an entity **Customer** with attributes `customer_id`, `name`, and `licence_number`. Every customer can have several phone numbers, so I gave **Customer** a simple attribute called `phone_number`. I connected **Car** and **Customer** with a relationship **Rents**, which I made **one-to-one**, since at the moment a customer picks up a car, it’s just the one customer and the one car. I also added a **RentalBranch** entity, for the pickup location, with just one attribute, `city` – I didn’t give it an id, since I don’t really need to look branches up individually.”

Find as many problems as you can with this description, and say how you would fix each one.

*Hint: there are (at least) four separate issues here – one about a multi-valued attribute, one about a missing key, one about a cardinality that does not match how car rentals actually work over time, and one about whether **RentalBranch** should be a weak entity.*

Solution.

- **Multi-valued attribute drawn as simple:** `phone_number` on **Customer** is described as holding several values per customer, but declared as a simple attribute. It should be a **multi-valued** attribute, e.g. `phone_numbers`.
- **Missing identifying attribute:** **RentalBranch** has only `city` and no id – but if two branches can be in the same city, or the branch needs to be referenced/looked up (e.g. from a rental record), it needs its own key, e.g. `branch_id`. (Whether it then needs to be a weak entity depends on whether it can be identified on its own – with a `branch_id` of its own it is a normal strong entity, not a weak one; the classmate’s instinct to skip giving it an id is exactly the mistake.)
- **Wrong cardinality:** **Rents** is declared **1:1** between **Car** and **Customer**. But over time, one customer can rent many different cars (on different occasions), and one car can be rented by many different customers (at different times) – the relationship should be **M:N**, with the pickup/return dates as attributes on the relationship (as in Exercise 1) to distinguish one rental occasion from another. A 1:1 reading only holds for a single instant in time, which is not what an ER model captures.
- **Missing entity/relationship for the branch link:** even after fixing **RentalBranch** to have a key, the description never says how **RentalBranch** connects to the rest of the model – e.g. a **Located_At** or **Picked_Up_From** relationship between **RentalBranch** and **Car** (which branch a car is normally kept at) is missing entirely and should be added.

5 Exercise 5: Designing the Library ER Model

A library wants a database to manage its collection and its members. Model the following requirements as a full ER diagram:

- The library holds books. Each book may exist as one or more physical copies on the shelf, and the library needs to track the status of each individual copy (e.g. whether it is currently available, on loan, or lost) separately – two copies of the same book can be in different states at the same time.
- Each book may have been written by one or more authors, and an author may have written several books.
- The library has members, who can borrow copies of books.
- For every loan, the library needs to record when the copy was borrowed, when it is due back, and when (if at all) it was actually returned.
- Books can belong to more than one genre (e.g. a book can be both “Fiction” and “Mystery”).
- Both books and authors need some basic descriptive information (title, publication year, an identifying number such as an ISBN for books; a name and birth year for authors). Members need a name, an email address, and the date they joined the library.

Draw a full ER diagram (on paper, in draw.io, or in Mermaid – see `figures/library-er-diagram.puml` in this exercise sheet’s own folder for a PlantUML source file you can adapt) that captures all of this. For every entity, list its attributes and mark which ones form the key (or partial key, for a weak entity), and note the kind of any attribute that is not simple. For every relationship, give its name, the entities it connects, its cardinality, and the participation constraint on each side. Think carefully about whether any entity here is a *weak entity*.

Hint: re-read the first bullet point above very carefully – it is telling you something specific about how copies of a book should be modelled.

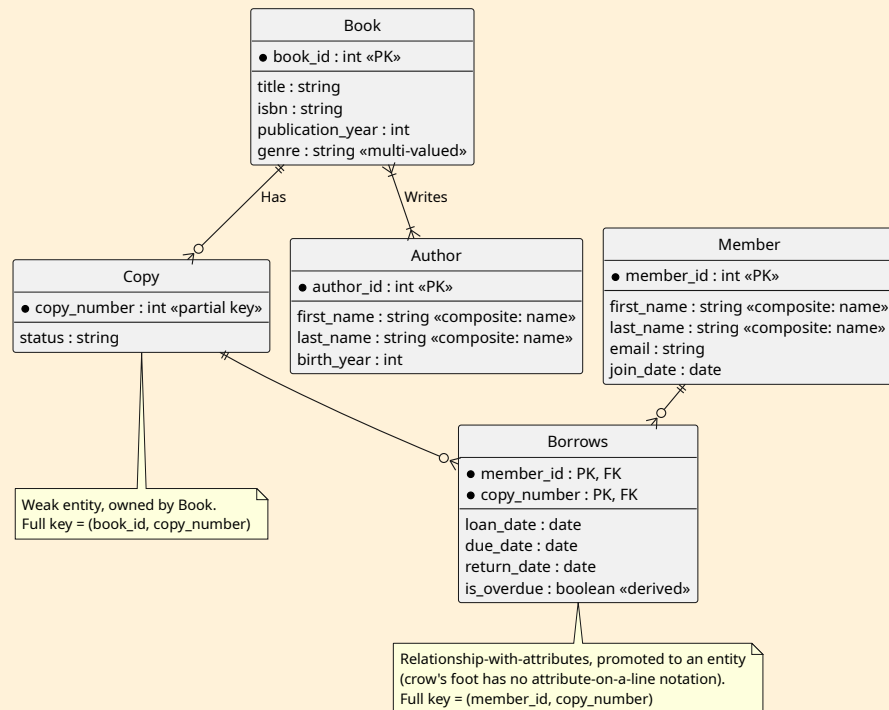
Solution.

Entities and attributes:

Entity	Attribute	Kind	Key?
<i>Book</i> (strong)	<i>book_id</i>	simple	<u>yes (PK)</u>
	<i>title</i>	simple	
	<i>isbn</i>	simple	
	<i>publication_year</i>	simple	
	<i>genre</i>	multi-valued	
<i>Copy</i> (weak, owned by <i>Book</i>)	<i>copy_number</i>	simple	<u>partial key</u>
	<i>status</i>	simple	
<i>Author</i> (strong)	<i>author_id</i>	simple	<u>yes (PK)</u>
	<i>name</i>	composite (<i>first_name</i> + <i>last_name</i>)	
	<i>birth_year</i>	simple	
<i>Member</i> (strong)	<i>member_id</i>	simple	<u>yes (PK)</u>
	<i>name</i>	composite (<i>first_name</i> + <i>last_name</i>)	
	<i>email</i>	simple	
	<i>join_date</i>	simple	
<i>Borrows</i> (promoted from a relationship-with-attributes – see below)	<i>member_id</i>	simple	<u>PK, FK</u>
	<i>copy_number</i>	simple	<u>PK, FK</u>
	<i>loan_date</i>	simple	
	<i>due_date</i>	simple	
	<i>return_date</i>	simple	
	<i>is_overdue</i>	derived	

Relationships:

Name	Entities	Card.	Participation	Attributes
<i>Writes</i>	<i>Book</i> – <i>Author</i>	<i>M:N</i>	<i>total on both sides</i>	–
<i>Has</i> (identifying)	<i>Book</i> – <i>Copy</i>	<i>1:N</i>	<i>Copy: total; Book: partial</i>	–
–	<i>Member</i> – <i>Borrows</i>	<i>1:N</i>	<i>Borrows: total; Member: partial</i>	–
–	<i>Copy</i> – <i>Borrows</i>	<i>1:N</i>	<i>Borrows: total; Copy: partial</i>	–



The diagram this table describes – also available as an editable PlantUML source at [figures/library-er-diagram.puml](#).

Notes:

- *Copy* is a **weak entity**: two copies of the same book are distinguished only by *copy_number*, which is not globally unique on its own (copy 1 of *book_id*=5 and copy 1 of *book_id*=9 are different rows) – exactly the same shape as *Topic* in Exercise 3. *Copy*'s full key is (*book_id*, *copy_number*). *Has* is its identifying relationship, and *Copy* has **total** participation in it (a copy cannot exist without its book); *Book* has **partial** participation (a book record could in principle exist with zero physical copies currently on the shelf, e.g. all lost).
- *genre* on *Book* is **multi-valued** (a book can have several genres) – this was the detail the hint pointed at together with the weak-entity point about *Copy*.
- *Borrows* began life as a relationship with attributes (*loan_date*, *due_date*, *return_date*) between *Member* and *Copy* – a loan is a fact about a particular member borrowing a particular copy on a particular occasion, not about *Member* or *Copy* alone. Crow's foot notation has no way to attach an attribute to a relationship line, though, so – exactly as taught on the "Same Fact, in Crow's Foot" slide – *Borrows* is **promoted to its own entity**, keyed by (*member_id*, *copy_number*) and linked to *Member* and *Copy* by two ordinary 1:N relationships instead of one M:N relationship. *is_overdue* is a **derived** attribute on *Borrows*, computed from *due_date* vs. today, and only meaningful while *return_date* is still empty.
- *Writes* has **total** participation on both sides here because every book in the library must

have at least one author on record, and every author on record must have written at least one book in the collection.

6 Exercise 6: A Larger Model – University

A university wants a database covering its programs, courses, semesters, teachers, and students. Model the following requirements as a full ER diagram:

- The university offers several programs (e.g. DAT, SWT, DVML). Each program is organized into semesters, numbered *within* the program (e.g. semester 2 of DVML is informally called “DVML2”, semester 5 of DAT is “DAT5”); a semester only makes sense together with the program it belongs to.
- A course is offered as part of one or more semesters, and those semesters can belong to different programs – e.g. the same “Databases” course might be part of both DAT’s semester 5 and SWT’s semester 3.
- A teacher teaches one or more courses. A course is taught by one or more teachers (at least one).
- Each semester has exactly one teacher assigned as its coordinator. A teacher coordinates at most one semester (possibly none).
- A student is enrolled in exactly one program.
- A student is registered for (“takes”) many courses.
- For every course a student takes, the student can have zero or more grades on record for that course, each with the date of the exam it was awarded for – a student can sit the exam for the same course more than once (e.g. after failing).

Draw a full ER diagram that captures all of this. For every entity, list its attributes and mark the key (or partial key, for a weak entity). For every relationship, give its name, the entities it connects, its cardinality, and the participation constraint on each side.

*Hint: three things need extra care here. (1) **Semester** is a weak entity, the same way **Topic** was in Exercise 3 – work out what it depends on. (2) One fact here cannot be modelled as a plain relationship with attributes, the same way **Borrows** could not in the lecture – and it needs something extra beyond just being promoted to an entity. Which fact, and why? (3) Look for a relationship here that is **1:1**, not **1:N** or **M:N**.*

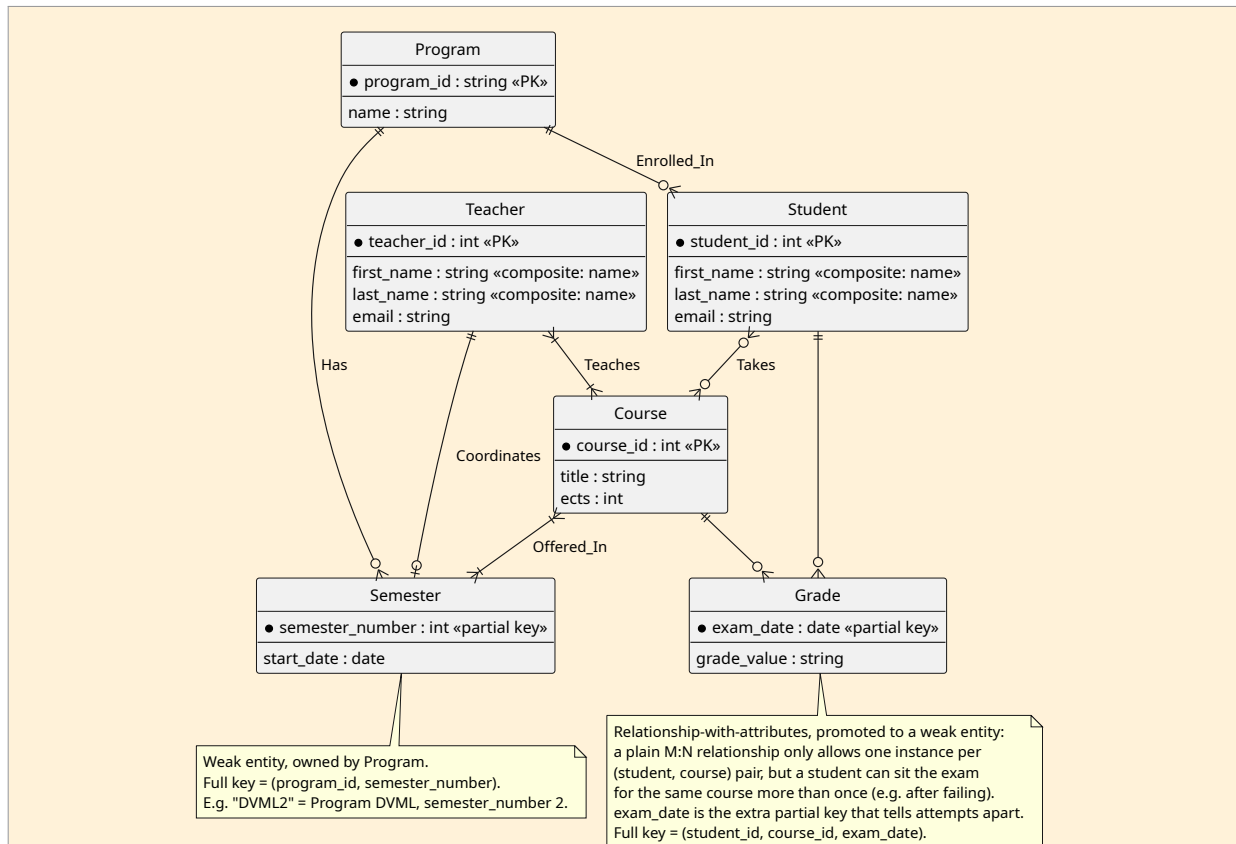
Solution.

Entities and attributes:

Entity	Attribute	Kind	Key?
Teacher (strong)	teacher_id	simple	<u>yes (PK)</u>
	name	composite (first_name + last_name)	
	email	simple	
Program (strong)	program_id name	simple simple	<u>yes (PK)</u>
Semester (weak, owned by Program)	semester_number	simple	<u>partial key</u>
	start_date	simple	
Course (strong)	course_id	simple	<u>yes (PK)</u>
	title	simple	
	ects	simple	
Student (strong)	student_id	simple	<u>yes (PK)</u>
	name	composite (first_name + last_name)	
	email	simple	
Grade (weak, owned by Student and Course; promoted from a relationship-with- attributes)	exam_date	simple	<u>partial key</u>
	grade_value	simple	

Relationships:

Name	Entities	Card.	Participation
Teaches	Teacher – Course	M:N	total on both sides
Coordinates	Teacher – Semester	1:1	Semester: total; Teacher: partial
Has (identifying)	Program – Semester	1:N	Semester: total; Program: partial
Enrolled_In	Program – Student	1:N	Student: total; Program: partial
Takes	Student – Course	M:N	partial on both sides
Offered_In	Course – Semester	M:N	total on both sides
–	Student – Grade	1:N	Grade: total; Student: partial
–	Course – Grade	1:N	Grade: total; Course: partial



The diagram this table describes – also available as an editable PlantUML source at [figures/university-er-diagram.puml](#).

Notes:

- **Semester is a weak entity:** *semester_number* only distinguishes semesters within the same program – semester 2 of DVML and semester 2 of DAT are different rows. Its full key is (*program_id*, *semester_number*), and *Has* is its identifying relationship – the same shape as *Topic* in Exercise 3, or *Copy* in Exercise 5.
- **Grade began life as a relationship with attributes** (*grade_value*, *exam_date*) between *Student* and *Course*, exactly like *Borrows* in Exercise 5 – so, following the same reasoning as the “Same Fact, in Crow’s Foot” slide, it is promoted to its own entity. Unlike *Borrows*, though, (*student_id*, *course_id*) alone is not enough as a key: the scenario explicitly allows a student to sit the exam for the same course more than once, so more than one *Grade* row can exist for the same (*student*, *course*) pair. *exam_date* is added as an extra **partial key** to tell repeated attempts apart – the same weak-entity trick as *Topic*’s *topic_number*, combined with the “promoted relationship” trick at the same time. *Grade*’s full key is therefore (*student_id*, *course_id*, *exam_date*).
- **Coordinates is a rare 1:1 relationship:** every semester requires exactly one coordinating teacher (*Semester*’s participation is total), but a teacher coordinates at most one semester and might coordinate none at all (*Teacher*’s participation is partial).

- *Student* sits on the “many” side of *Enrolled_In* (a 1:N relationship from *Program*), but *Student* is **not** a weak entity: it already has its own key, *student_id* – the same distinction Exercise 3’s Question 4 asked about *Course* and *Professor*.
- There is no *semester* attribute directly on *Takes*: which semester(s) a course belongs to is already captured by *Offered_In*, so repeating it on *Takes* would just be redundant, unsynchronized data.

7 Exercise 7: Model a Domain of Your Choice

Think of a domain – it might be your semester project. Model the entities and relations in that domain as an ER diagram.

Try to include:

- **At least 4 entity types.**
- **At least 1 weak entity** (with its identifying relationship and partial key clearly marked).
- **At least 1 M:N relationship.**
- For every entity: list its attributes, mark the key (or partial key), and note the kind (simple/composite/multi-valued/derived) of any attribute that is not simple.
- For every relationship: give its name, the entities it connects, its cardinality, and the participation constraint on each side.

There is nothing to submit for this exercise – you can draw it on paper or use a tool, such as draw.io, Mermaid, or PlantUML, whichever you find easiest, and use it to self-check your understanding.

Self-check checklist:

- ☐ At least 4 entity types
- ☐ At least 1 weak entity, with partial key and identifying relationship marked
- ☐ At least 1 M:N relationship
- ☐ Every attribute's kind is clear (simple attributes need no label; composite/multi-valued/derived ones should be marked)
- ☐ Every relationship has a cardinality and a participation constraint on each side

Solution.

*This exercise is **open-ended** – every domain (semester project or otherwise) is different, so there is no single fixed solution to check against. Compare your own diagram against the checklist above: in particular, make sure your weak entity is genuinely weak (it cannot be identified by its own attributes alone and depends on an owner entity), and that your M:N relationship is genuinely M:N and not actually a 1:N relationship in disguise. If anything feels uncertain, bring it to a TA session for a second opinion.*

AI Disclosure

Claude, a large language model developed by Anthropic, was used to assist with structuring and grammar checking of this material. All generated text was critically reviewed, edited, and verified by the author.